



算法设计与分析

讲授内容：递归与分治法

2025年11月19日

主方法（复习）

主方法适用于递归形式： $T(n) = aT(n/b) + f(n)$ ，
 $a \geq 1, b > 1$, f 为正的渐进函数

1. $f(n) = O(n^{\log_b a - \varepsilon})$, 常量 $\varepsilon > 0$ 。

解： $T(n) = \Theta(n^{\log_b a})$

2. $f(n) = O(n^{\log_b a})$

解： $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$

3. $f(n) = \Omega(n^{\log_b a + \varepsilon})$, 常量 $\varepsilon > 0$

$f(n)$ 增长比多项式 $n^{\log_b a}$ 快（快 n^ε 倍）

并且 $f(n)$ 满足规定条件

$af(n/b) \leq cf(n)$, 常量 $c < 1$ 。

解： $T(n) = \Theta(f(n))$ 。

- 递归
 - 递归的概念
 - 递归方程举例
 - 递归的优缺点
- 分治法
 - 分治法的基本思想
 - 分治法求解问题的基本步骤
 - 分治法求解问题举例

- 递归函数：用函数自身给出定义的函数
- 递归函数的二个要素：
 - 边界条件与递归方程
 - 举例：阶乘函数的递归方程？
- 递归算法：直接或间接调用自身的算法

- 优点：
 - 算法简明
 - 正确性易证明
 - 是分析、设计的有力工具
- 缺点：
 - 执行效率不高
 - 堆栈空间耗费



大事化小，
小事化了

基本思想： 将一个难以直接解决的大问题，分割成规模较小的相同子问题，以便各个击破，分而治之。

举例：求幂

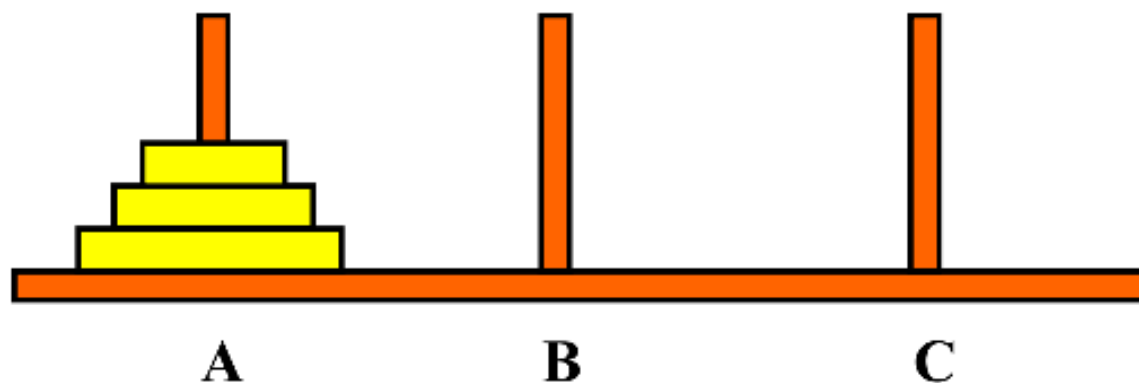
问题：计算 a^n , $n \in \mathbb{N}$.

简单算法： $\Theta(n)$.

1. 分解为几个子问题，子问题规模
 - 最好使子问题的规模大致相同
2. 子问题如何求解
3. 合并原问题的解？
4. 时间复杂度

举例：Hanoi塔问题

Hanoi塔问题



$$T(n) = 2 T(n-1) + 1, \quad T(1) = 1, \quad \text{解得} \quad T(n) = 2^n - 1$$

1. 可以分解为若干个规模较小的相同问题
2. 分解出的各个子问题是相互独立的，子问题之间不包含公共的子问题
3. 问题规模缩小到一定的程度容易解决
4. 利用该问题分解出的子问题的解可以合并为原问题的解

1. 分解：将问题分解成子问题

- 子问题：规模较小、相互独立、与原问题相同

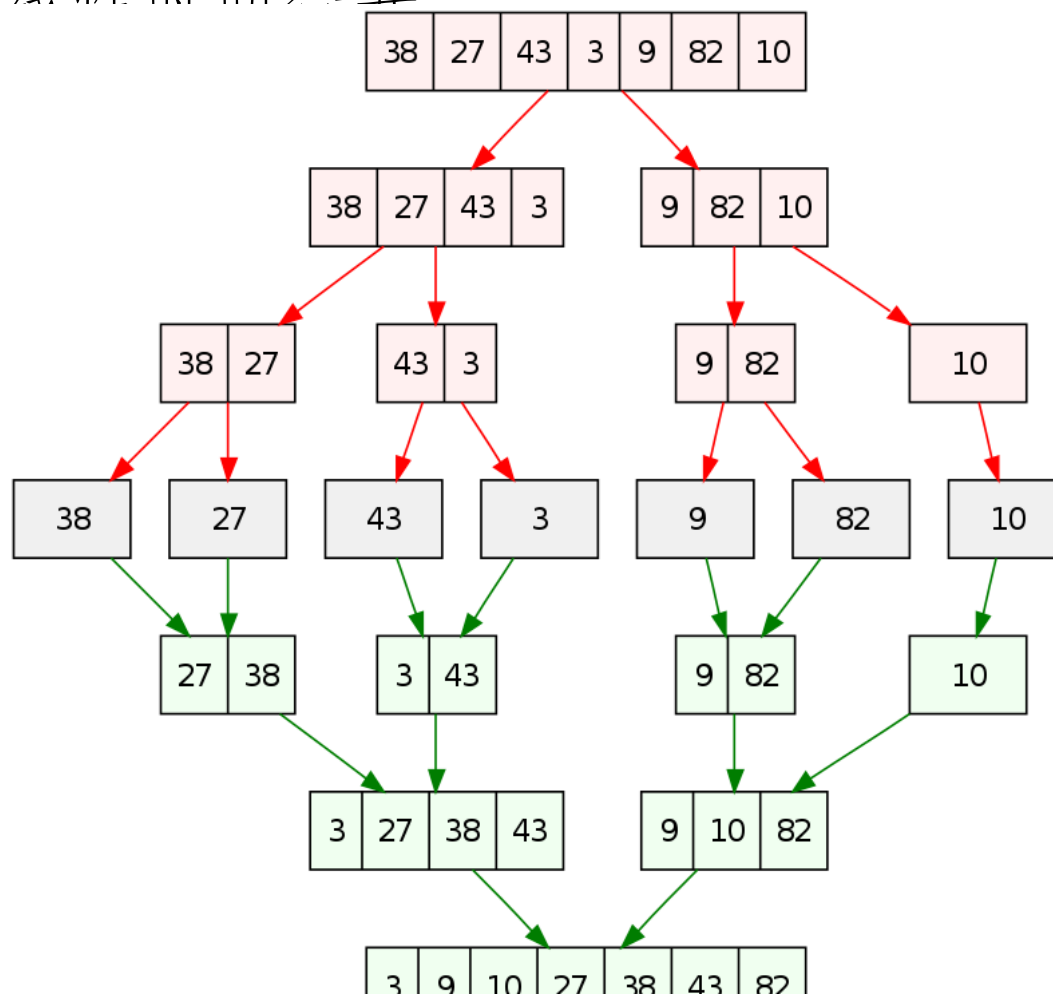
2. 求解：递归地解决子问题

- 若子问题规模较小且容易被解决则直接解
- 否则继续分解为更小的子问题

3. 合并：组合子问题的答案

举例：归并排序

1. 分解：显而易见。
2. 解决：递归地对两个子数组排序。
3. 组合：线性时间合并



举例：归并排序

1. 分解：显而易见。
2. 解决：递归地对两个子数组排序。
3. 组合：线性时间合并。

$$T(n) = 2T(n/2) + \Theta(n)$$

子问题数目 子问题规模 分解和组合工作

举例：求解时间复杂度

1. 设阈值 $n_0 = 1$ 时，时间耗费为 7
2. 分解及合并成原问题的时间为 $5n^2$

$$T(n) = \begin{cases} 7 & n = 1 \\ 2T\left(\frac{n}{2}\right) + 5n^2 & n > 1 \end{cases}$$

举例：二分查找

在排序的数组中查找元素

1. 分解：检查中间元素
2. 解决：递归查找一个子数组
3. 组合：显而易见

例子：查找 9

3 5 7 8 9 12 15

举例：二分查找

- 在排序的数组中查找元素
 1. 分解：检查中间元素
 2. 解决：递归查找一个子数组
 3. 组合：显而易见
- 例子：查找 9

3 5 7 8 9 12 15

举例：二分查找

在排序的数组中查找元素

1. 分解：检查中间元素
2. 解决：递归查找一个子数组
3. 组合：显而易见

例子：查找 9

3 5 7 8 9 12 15

举例：二分查找

在排序的数组中查找元素

1. 分解：检查中间元素
2. 解决：递归查找一个子数组
3. 组合：显而易见

例子：查找 9

3 5 7 8 9 12 15

举例：二分查找

在排序的数组中查找元素

1. 分解：检查中间元素
2. 解决：递归查找一个子数组
3. 组合：显而易见

例子：查找 9

3 5 7 8 9 12 15

举例：二分查找

在排序的数组中查找元素

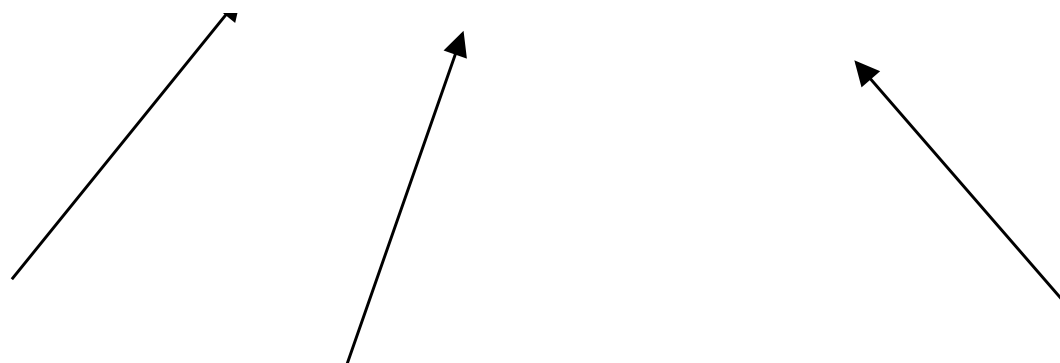
1. 分解：检查中间元素
2. 解决：递归查找一个子数组
3. 组合：显而易见

例子：查找 9

3 5 7 8 9 12 15

举例：二分查找的递归

$$T(n) = 1T(n/2) + \Theta(1)$$



子问题数

子问题大小

分解和组合
工作

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1 \Rightarrow T(n) = \Theta(\lg n).$$

举例：硬币问题

问题描述：水果店老板共有 n 枚硬币，其中一枚是假币，假币重量略轻。目前只有一个天平，请设计一种算法，使老板用最少的次数找到该枚假币。

解法1：顺序搜索

解法2：二分搜索

解法3：三分搜索

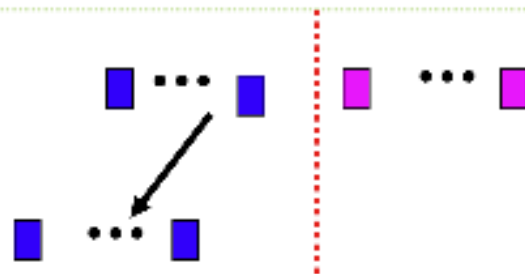
举例：硬币问题

问题描述：水果店老板共有 n 枚硬币，其中一枚是假币，假币重量略轻。目前只有一个天平，请设计一种算法，使老板用最少的次数找到该枚假币。

二分搜索应用

算法设计2：二分搜索

- 1) 将 n 枚硬币平均分为2堆；
- 2) 放入天平两端，留下重量轻的一堆，转1)，直到剩下1枚硬币；



二分搜索技术的应用，最坏情况下的时间复杂性为 $O(\log_2 n)$

进一步改进？

举例：硬币问题

问题描述：水果店老板共有 n 枚硬币，其中一枚是假币，假币重量略轻。目前只有一个天平，请设计一种算法，使老板用最少的次数找到该枚假币。

二分搜索-拓展

算法设计3：三分搜索

- 1)将 n 枚硬币平均分为3堆；
- 2)将其中2堆放入天平两端。若重量相等，则留下第3堆；
否则，留下重量轻的一堆，转1),直到剩下1枚硬币为止。

最坏情况下的时间复杂性为 $O(\log_3 n)$ 。

举例：整数划分问题

- 整数划分问题：
- 将正整数 n 表示成一系列正整数之和。求正整数 n 的不同划分个数 $p(n)$

举例：整数划分问题

- 整数划分问题：
- 将正整数 n 表示成一系列正整数之和。求正整数 n 的不同划分个数 $p(n)$

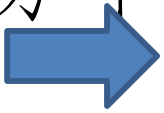
整数划分问题

将正整数 n 表示成一系列正整数之和：

$$n = n_1 + n_2 + \cdots + n_k \quad (n_1 \geq n_2 \geq \cdots \geq n_k \geq 1)$$

正整数 n 的这种表示称为正整数 n 的划分。求正整数 n 的不同划分个数 $p(n) = ?$

举例：整数划分问题

- 整数划分问题：
- 将正整数 n 表示成一系列正整数之和。求正整数 n 的不同划分个数 $p(n)$
- 没有思路？  以 $n=6$ 为例分析

6划分

正整数 $n=6$ 有如下11种不同的划分：

6	5+1	4+1+1 4+2	3+1+1+1 3+2+1 3+3	2+1+1+1+1 2+2+1+1 2+2+2	1+1+1+1+1+1
---	-----	--------------	-------------------------	-------------------------------	-------------

考虑增加一个自变量：最大加数 n_1 不大于 m 的划分个数记为 $q(n, m)$ 。

举例：整数划分问题

- 整数划分问题：
- 将正整数 n 表示成一系列正整数之和。求正整数 n 的不同划分个数 $p(n)$

整数划分

增加一个自变量：最大加数 n_1 不大于 m 的划分个数记为 $q(n, m)$ 。

$$1) n = 1, m = 1 \quad q(1, 1) = 1$$

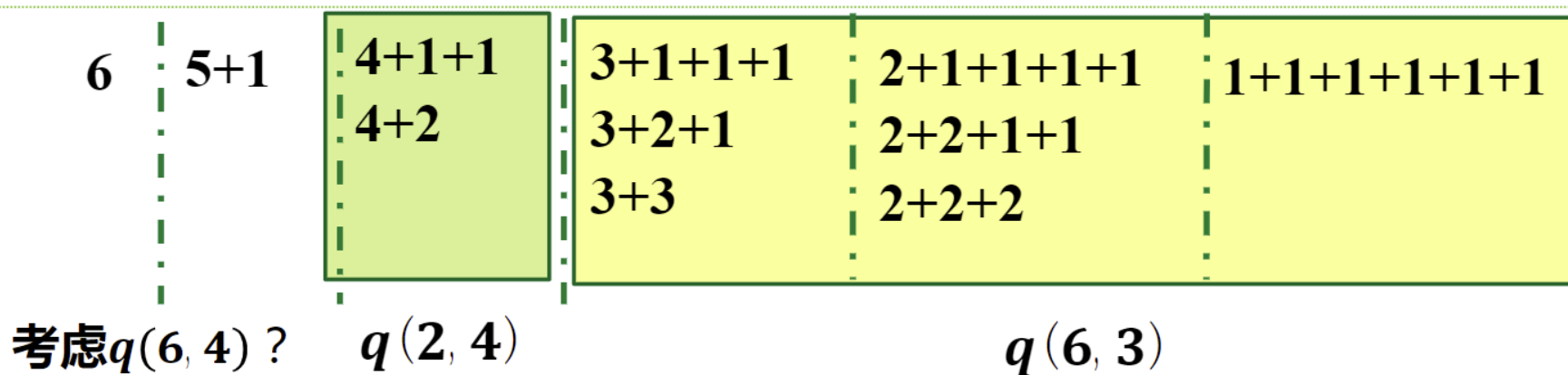
$$2) n < m \quad q(n, m) = q(n, n)$$

$$3) n = m \quad q(n, m) = 1 + q(n, n - 1)$$

举例：整数划分问题

- 整数划分问题：
- 将正整数 n 表示成一系列正整数之和。求正整数 n 的不同划分个数 $p(n)$

递归方程



递归方程： $q(n, m) = q(n, m - 1) + q(n - m, m)$

举例：整数划分问题

- 整数划分问题：
- 将正整数 n 表示成一系列正整数之和。求正整数 n 的不同划分个数 $p(n)$

整数划分

$$q(n, m) = \begin{cases} 1 & n = 1, m = 1 \\ q(n, n) & n < m \\ 1 + q(n, n - 1) & n = m \\ q(n, m - 1) + q(n - m, m) & n > m > 1 \end{cases}$$

正整数 n 的划分数 $p(n) = q(n, n)$

举例：矩阵相乘

输入： $A = [a_{ij}], B = [b_{ij}].$
输出： $C = [c_{ij}] = A \cdot B.$ } $i, j = 1, 2, \dots, n.$

$$\begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{bmatrix}$$

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$$

举例：矩阵相乘标准算法

```
for  $i \leftarrow 1$  to  $n$ 
  do for  $j \leftarrow 1$  to  $n$ 
    do  $c_{ij} \leftarrow 0$ 
      for  $k \leftarrow 1$  to  $n$ 
        do  $c_{ij} \leftarrow c_{ij} + a_{ik} \cdot b_{kj}$ 
```

运行时间 = $\Theta(n^3)$

举例：矩阵相乘分而治之算法

思想:

$n \times n$ 矩阵 = 2×2 个 $(n/2) \times (n/2)$ 子矩阵:

$$\begin{bmatrix} r & | & s \\ \hline t & | & u \end{bmatrix} = \begin{bmatrix} a & | & b \\ \hline c & | & d \end{bmatrix} \cdot \begin{bmatrix} e & | & f \\ \hline g & | & h \end{bmatrix}$$

$$C = A \cdot B$$

$$\begin{aligned} r &= \\ ae+bg \\ s &= af \\ &+bh \\ t &= \end{aligned}$$

8 个 $(n/2) \times (n/2)$ 子矩阵相乘
4 个 $(n/2) \times (n/2)$ 子矩阵相加

$$ce+dg$$

$$u = cf$$

举例：矩阵相乘分而治之算法分析

$$T(n) = 8T(n/2) + \Theta(n^2)$$

子矩阵数目 子矩阵范围 子矩阵相加

$$n^{\log_b a} = n^{\log_2 8} = n^3 \Rightarrow \text{CASE 1} \Rightarrow T(n) = \Theta(n^3).$$

比普通的算法没有什么改进.

举例：矩阵相乘-Strassen's 思想

- 仅用7个递归乘完成 2×2 矩阵相乘

$$P1 = a \cdot (f - h)$$

$$P2 = (a + b) \cdot h$$

$$P3 = (c + d) \cdot e$$

$$P4 = d \cdot (g - e)$$

$$P5 = (a + d) \cdot (e + h)$$

$$P6 = (b - d) \cdot (g + h)$$

$$P7 = (a - c) \cdot (e + f)$$

$$r = P5 + P4 - P2 + P6$$

$$s = P1 + P2$$

$$t = P3 + P4$$

$$u = P5 + P1 - P3 -$$

$$P7$$

7 乘, 18 加/减.

注意: 不依赖于乘法的交换性!

举例：矩阵相乘-Strassen's 思想

- 仅用7个递归乘完成 2×2 矩阵相乘

$$P1 = a \cdot (f - h)$$

$$P2 = (a + b) \cdot h$$

$$P3 = (c + d) \cdot e$$

$$P4 = d \cdot (g - e)$$

$$P5 = (a + d) \cdot (e + h)$$

$$P6 = (b - d) \cdot (g + h)$$

$$P7 = (a - c) \cdot (e + f)$$

$$r = P5 + P4 - P2 + P6$$

$$= (a + d)(e + h)$$

$$+ d(g - e) - (a + b)h$$

$$+ (b - d)(g + h)$$

$$= ae + ah + de + dh$$

$$+ dg - de - ah - bh$$

$$+ bg + bh - dg - dh$$

$$= ae + bg$$

举例：矩阵相乘Strassen's 思想

1. **分解**: 将 A 和 B 划分成 $(n/2) \times (n/2)$ 的子矩阵. 用 $+$ and $-$ 组合成结果.
2. **解决**: 递归的进行 7 个 $(n/2) \times (n/2)$ 子矩阵相乘.
3. **组合**: 对 $(n/2) \times (n/2)$ 子矩阵进行 $+$ 和 $-$ 运算得到 C .

$$T(n) = 7 T(n/2) + \Theta(n^2)$$

举例：矩阵相乘Strassen算法分析

$$T(n) = 7 T(n/2) + \Theta(n^2)$$

$$n^{\log_b a} = n^{\log_2 7} \approx n^{2.81} \Rightarrow \text{CASE 1} \Rightarrow T(n) = \Theta(n^{\log 7}).$$

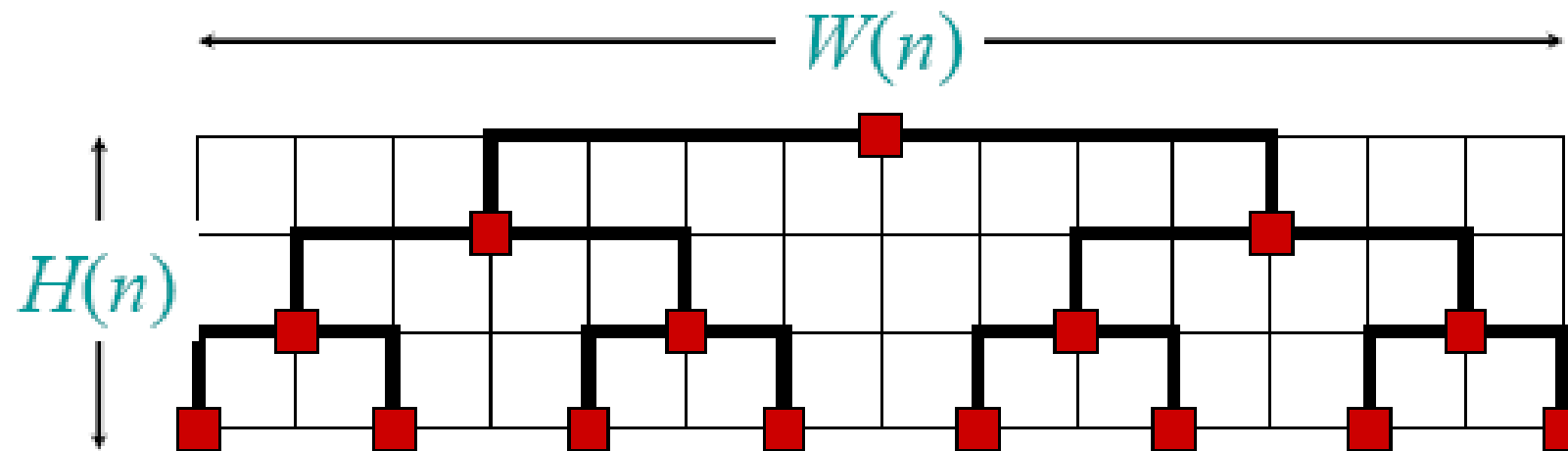
2.81 看起来和3差别不大, 但是因为区别是在指数项, 所以对运行时间的影响是很明显的。实际上, Strassen算法在今天普通的计算机上运行时, 在 $n \geq 32$ 超过普通的算法。

至今为止最好的 (仅仅是理论上): $\Theta(n^{2.376...})$.

举例：VLSI 布局

VLSI layout (Very Large Scale Integration, 超大规模集成电路布局问题)

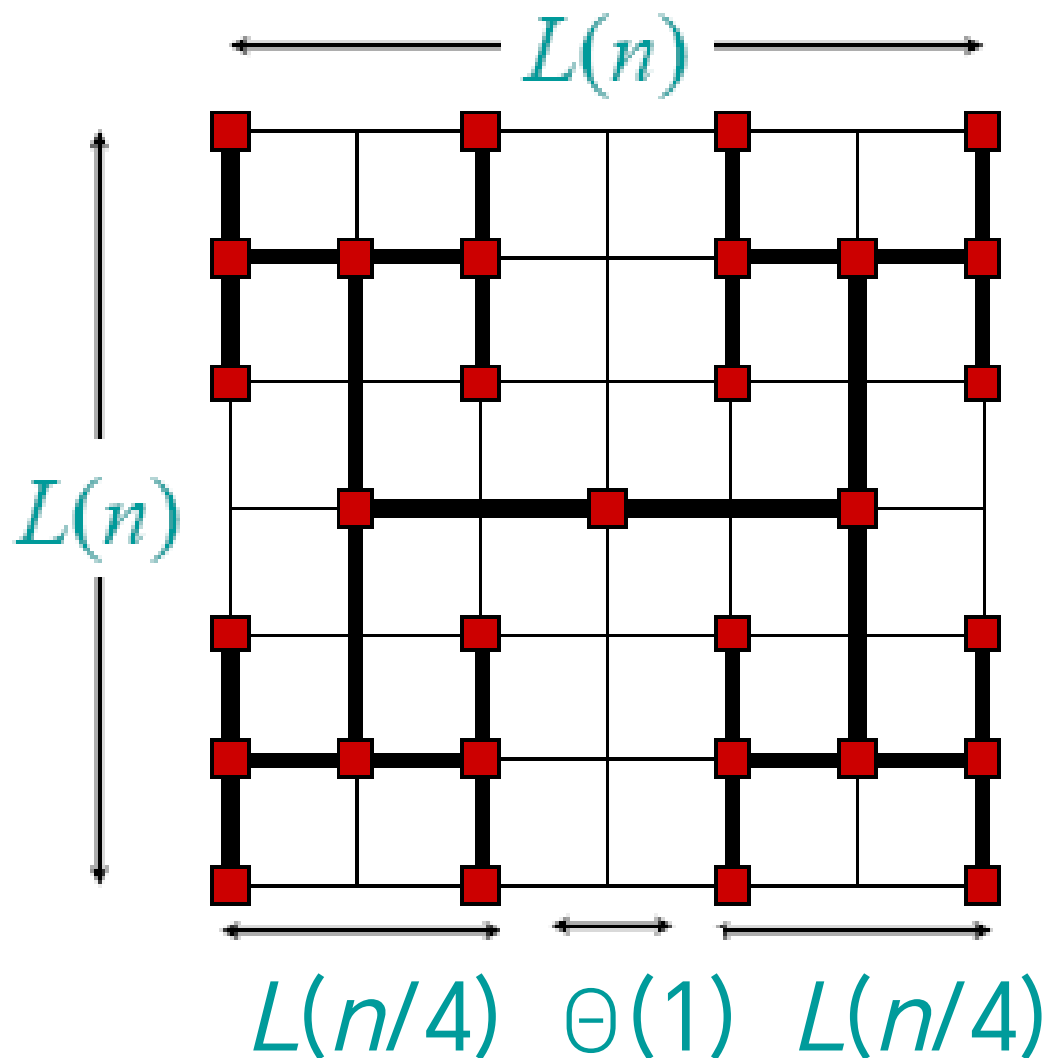
问题: 用最小的面积将一棵有 n 个叶子的完全二叉树嵌入网格。



$$\begin{aligned} H(n) &= H(n/2) + \Theta(1) & W(n) &= 2W(n/2) + \Theta(1) \\ &= \Theta(\lg n) & &= \Theta(n) \end{aligned}$$

$$\text{面积} = \Theta(n \lg n)$$

举例：VLSI 布局 H-树嵌入



$$\begin{aligned} L(n) &= 2L(n/4) + \Theta(1) \\ &= \Theta(\sqrt{n}) \end{aligned}$$

面积 = $\Theta(n)$

- 分而治之仅仅是算法设计中强大的设计技术之一。
- 分而治之算法可以用递归和主方法进行分析。
- 能得到更加高效的算法。

- 1. 求解递归式
 - a. $T(n) = 2T(n/4) + 1$
 - b. $T(n) = 2T(n/4) + n$
 - c. $T(n) = T(n/2) + \Theta(1)$
 - d. $T(n) = 2T(n/4) + n^2$
- 2. Hanoi塔问题: 在Hanoi这个地方有一个寺庙, 这里有3根柱子和n个大小不同的金碟子。每个碟子有一个孔可以穿过。所有的碟子都放在第一个柱子上, 而且按照从上到下碟子的大小依次增大的顺序摆设, 如下图所示。现在, 假定寺庙里的僧侣要移动这些碟子, 将它们从最左边移动到最右边的柱子上。不过移动的规则如下: **a)** 每次只能从一个柱子的最上面移动一个碟子到另外一个柱子上。**b)** 不能将大碟子放到小碟子的上面。



合肥工业大学
HEFEI UNIVERSITY OF TECHNOLOGY

Q/A?



合肥工业大学数据挖掘与智能计算“千人计划”研究团队
HFUT Data Mining and Intelligent Computing Laboratory